

HireMap

A Distributed Pipeline for Real-Time Job Market Intelligence

Asmita Singh

Abstract

In the first term, I built a fault-tolerant distributed pipeline that pulls job postings from several sources at once, removes duplicates, and returns results within a set deadline. It's a solid engine for collecting job data reliably. Phase 2 builds on it to make that data actually useful.

There are two parts. The first is keeping the data current: adding more sources and switching from one-off searches to continuous, scheduled crawling, with checks that confirm a listing is still live before it's shown. This also gives the pipeline's fault tolerance a real, ongoing workload to handle. The second is making sense of each posting. Job descriptions are long and vague, so I'll add a language-model step that reads each one and pulls out what matters: the skills that are actually required versus the ones that are just nice to have, the real seniority level, whether the role is genuinely remote, and the salary even when it isn't spelled out. The goal is simple: show jobs that are current, and show them as clear summaries instead of full descriptions a person has to wade through.

1. Introduction

The first term answered a systems question: Can a pipeline gather job data from unreliable sources reliably and within a deadline? It can, and that's what I built. Phase 2 picks up from there with a more practical goal, one that comes straight from my own experience looking for a job.

Two things slow a job search down. Listings go out of date; you click into something that looks new and find it was filled last week. And the postings that are still open take real effort to read; the important details are buried in long, generic descriptions. Phase 2 tackles both, and it does it by leaning on the pipeline I already built in phase I. The fault tolerance and deduplication from Term 1 are exactly what make it practical to crawl a lot more sources, continuously, without the data getting lost or doubled up. So that work finally gets put to use on a bigger, ongoing load. On top of that steady supply of current listings, a language model does the reading-and-summarizing that I would otherwise be doing by hand.

2. What I am Solving

Phase 2 comes down to two problems.

Keeping listings current: A job tool is only useful if what it shows is still open. That means moving from search-on-demand to crawling on a schedule, across more sources, and re-checking older listings to see whether they are still live: dropping or flagging the ones that aren't. The pipeline has to stay correct and stable as the number of sources grows.

Making postings readable: Raw job descriptions are unstructured and padded. I want to turn each one into a short, structured summary required versus preferred skills, seniority,

whether it's really remote, and salary. So I can compare roles at a glance instead of reading every description in full.

Underneath these sit a few systems pieces: adding a source shouldn't mean rewriting the core (so I will build a pluggable adapter), crawling has to respect each source's rate limits, liveness checks have to be efficient, and the language-model work needs to run as part of the pipeline, as more workers on the queue rather than as a single slow step.

3. Why It's Worth Doing

When you're job hunting, the hard part isn't finding postings anymore, there are plenty. It's the time wasted on the wrong ones: roles that are already closed, and descriptions that take a few minutes each just to figure out what's actually being asked. If the listings are kept current and each one is shown as a clean summary, that wasted time mostly goes away, and a long noisy list becomes a short readable one (otherwise can be achieved using extensions). That's the version of "job market intelligence" the project was always aiming at, and it's something I'd genuinely use.

4. Why Existing Tools Don't Cover This

Tool	What it does	What's missing
LinkedIn / Indeed	Big centralized listings	Old postings linger; descriptions shown as-is; no combined decoded view across sources
Generic aggregators	Pull from several boards	Don't check whether listings are still live; don't interpret the content
Searching by hand	Check each board yourself	Slow and repetitive, and no help reading or vetting what you find

Nothing I have found combines listings that are kept current and verified with automatic summarizing of what each posting actually wants. That's the gap Phase 2 fills.

5. How I will Build It

Phase 2 extends the Term 1 system: the Flask API, RabbitMQ broker, worker pool, PostgreSQL, and React dashboard. The new pieces:

Pluggable source adapters: I will refactor the sources so each one sits behind a shared interface, which means adding a new source is just writing one adapter, not touching the core. Each source is isolated, so if one breaks or gets rate-limited, the others keep going. The sources I am targeting are free and aboveboard: Greenhouse, Lever, Remotive, Arbeitnow, USAJobs. (I am still leaving out Indeed and LinkedIn, for the same legal and technical reasons as Term 1.)

Continuous crawling and liveness checks: A scheduler crawls the sources on a recurring basis instead of only when someone searches, so the data stays fresh. A lightweight

checker revisits stored listings now and then to see if they're still up, marks dead ones as expired, and records when each listing was first and last seen, so the dashboard only shows roles that are actually open.

The language-model layer: A new kind of worker takes postings off the queue and uses a language model to pull structured fields out of the description: required vs preferred skills, seniority, remote/hybrid/onsite, and salary (including a sensible estimate when it's left out). Since it's just another worker on the existing queue, it scales the same way the rest of the system does and recovers from failures the same way. To keep cost and speed reasonable, I will use a small, efficient model and process postings as they come in rather than all at once.

Dashboard: Each job shows up as a decoded card: skills, seniority, work arrangement, salary, and a "still active / posted N days ago" label, so I am reading summaries instead of full descriptions. There's also a simple market view (most-requested skills, hiring activity) built from the data that accumulates over time.

6. Tech Stack (on top of Term 1)

Piece	Technology	Why
Scheduler	APScheduler	Lightweight, plugs into the existing Flask service
Source adapters	Python (shared interface)	Keeps sources isolated and easy to add
Liveness checker	Python worker	Reuses the worker/queue model I already have
Interpretation	Small/efficient LLM (API or local)	Turns messy descriptions into structured fields
Analytics	Pandas (+ existing Recharts)	Skill and market summaries on the dashboard

7. Bi-Weekly Plan

I am building the supply-and-freshness side first, then putting the language-model layer on top.

Weeks	Deliverable
1–2	Refactor the existing sources behind a pluggable, isolated adapter interface.
3–4	Add the scheduler for continuous crawling; make it respect each source's rate limits.
5–6	Bring in 2–3 new sources (Greenhouse, Lever, Remoteive) through the adapter.

7–8	Build the liveness checker and lifecycle tracking (first/last seen, expiry); dashboard shows only live listings.
9–10	First interpretation worker: LLM pulls skills, seniority, and work arrangement into structured fields, running through the queue.
11–12	Extend interpretation (salary estimate, required vs preferred); build the decoded job-card UI.
13–14	Add the market summary view (top skills, hiring activity); improve interpretation quality.
15–16	End-to-end testing, evaluation, documentation, and demo. (Stretch: start resume-based matching if there's time.)

8. How to Measure Success

- **Freshness:** roles shown as active are actually still live, and expired ones get caught and removed within a reasonable re-check window: a clear drop in stale results compared to Term 1.
- **Interpretation quality:** the fields the model pulls out (skills, seniority, work arrangement, salary) match human judgment on a sample at a target accuracy, and the summaries are clearly faster to read than the originals.
- **Scale and resilience:** the pipeline keeps crawling steadily across 6–8 isolated sources, one source failing doesn't drag down the rest, and the interpretation work runs as resilient workers on the queue.

9. Conclusion

Phase 2 takes HireMap from listing jobs to actually understanding them. By adding more sources, crawling continuously, checking that listings are still live, and using a language model to turn each posting into a clean summary, it becomes something I would genuinely use in my own search while giving the pipeline I built in Term 1 a bigger, real, ongoing job to do. The freshness-and-interpretation core is what I am committing to this term; resume-based matching and the heavier scaling work a continuous pipeline will eventually need are the natural next steps after that.